LLM-Prolog on VFR Hardware — Inline Intelligence Specification v1.0

Neural-Symbolic Computation as Native Batch Operations on Integer ALU Architecture

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Overview

The LLM is not a service. It is not an API call. It is not a separate system that processes requests and returns results. The LLM is a batch operation that runs inline with entity processing, Prolog evaluation, UtilityAI scoring, and rendering. It occupies the same cores, processes the same VFR format, and produces output that feeds directly into the next batch pass.

When a game entity needs new behavior, the pipeline does not pause to "ask the AI." The pipeline includes an LLM batch pass that generates new Prolog rules as VFR data, which the next Prolog batch pass immediately evaluates. Intelligence is not a separate phase. It is a transform in the pipeline, same as fact generation or behavior scoring.

---

2. Everything Is VFR

Every data type in the LLM-Prolog system is `[V: i32, R: i8]`.

2.1 Text as VFR

Characters are VFR pairs where V is the Unicode codepoint and R is the script identifier.

Character: [V: i32, R: i8]

V = Unicode codepoint

R = script family

R values:

0 = Latin/ASCII

1 = CJK Unified (Chinese)

2 = Hiragana (Japanese)

3 = Katakana (Japanese)

4 = Hangul (Korean)

5 = Cyrillic

6 = Arabic

7 = Hebrew

8 = Devanagari

9 = Thai

10 = Greek

11 = Emoji

…up to 127 script families

A string is a batch of characters:

“Hello ??”

[F=text, count=8, depth=1]

[72, 0] H, Latin

[101, 0] e, Latin

[108, 0] l, Latin

[108, 0] l, Latin

[111, 0] o, Latin

[32, 0] space, Latin

[19990, 1] □, CJK

[30028, 1] □, CJK

Properties:

- Fixed 5 bytes per character. O(1) indexing. No multi-byte scanning.
- Script detection is one byte read. No Unicode range lookup.
- Filtering by script is a batch filter on R. One comparison per character.
- Font selection reads R for atlas, V for glyph. No fallback chain.
- Sorting, searching, and comparing are integer operations on V.

2.2 Terms as VFR

A Term from the tokenizer is a VFR batch at depth 5:

Term batch:

[F=term, count=N, depth=5]

Each Term:

[term_type, 0] pair 0: TermType enum as i32

[value, 0] pair 1: vocabulary index or literal value

[source_id, 0] pair 2: provenance — which source file/document

[offset, 0] pair 3: provenance — byte offset in source

[version, 0] pair 4: provenance — source material version

TermType values:

TermType enum (i32):

0 = container function, struct, sentence

1 = delimiter_open ({ [”

2 = delimiter_close) }] ”

3 = separator , ; . : |

4 = name identifier: max, allocator, serpent

5 = kind category: fn, struct, noun, verb

6 = type_ref type: i32, bool, []u8

7 = keyword control: if, while, return

8 = operator + - * == >

9 = number integer literal

10 = text_literal string content (pointer to text batch)

11 = predicate Prolog predicate name (as integer ID)

12 = rule Prolog rule reference

13 = fact Prolog fact assertion

14 = variable Prolog unification variable

15 = reference entity/index pointer

16 = vector2 spatial point (2 values)

17 = rectangle spatial box (4 values)

18 = entity entity reference

19 = transform positional transform

Every Term carries provenance in its depth. Source ID, byte offset, and version travel with the Term through the entire pipeline. They are never discarded. A fact derived from a Term inherits the Term's provenance.

2.3 Facts as VFR

A Prolog fact is a batch entry where F is the predicate ID:

Fact batch for predicate "has_weapon" (ID=47):

[F=47, count=500, depth=6]

Each fact:

[arg_0, 0] pair 0: first argument (entity_id)

[arg_1, 0] pair 1: second argument (weapon_type)

[source_id, 0] pair 2: provenance source

[offset, 0] pair 3: provenance offset

[version, 0] pair 4: source version

[confidence, 0] pair 5: confidence level (0=speculative, 1=inferred, 2=stated, 3=verified)

The predicate name is F. The arguments are pairs 0-1. The provenance is pairs 2-5. Every fact in the entire knowledge base has this structure. Different predicates have different F values and potentially different argument counts, but the provenance fields are always present at the same depth offsets.

2.4 Rules as VFR

A Prolog rule is a batch describing the head predicate and the body predicates:

Rule batch:

[F=rule, count=N, depth=3]

Each rule entry:

[head_predicate_id, 0] pair 0: what this rule proves

[body_batch_addr, 0] pair 1: pointer to batch of body predicates

[body_count, 0] pair 2: number of predicates in body

The body is itself a batch of predicate references:

Body batch:

[F=rule_body, count=3, depth=2]

Each body predicate:

[predicate_id, 0] pair 0: which predicate to check

[arg_pattern_addr, 0] pair 1: pointer to argument matching pattern

Rules are data. Changing behavior means writing new rule batches. The FPGA processes them identically to any other batch.

2.5 LLM Weights as VFR

Neural network weights are VFR integers following the CKS-MATH-134 integer training approach:

Weight matrix batch:

[F=weights, count=rows*cols, depth=1]

Each weight:

[weight_value: i32, quantization_remainder: i8]

Activations during inference:

Activation batch:

[F=activations, count=layer_width, depth=1]

Each activation:

[activation_value: i32, remainder: i8]

The forward pass is matrix multiply-accumulate: load weight row, multiply element-wise with activation vector, accumulate. MUL, ADD, SHR on integer batches. Same instructions that do everything else.

2.6 Knowledge Base as VFR

The complete knowledge base is a stream of fact batches in DDR3:

KB stream:

[F=47, count=500, depth=6] [facts about has_weapon…]

[F=23, count=300, depth=6] [facts about in_range…]

[F=88, count=100, depth=6] [facts about health_critical…]

[F=12, count=200, depth=6] [facts about has_item…]

…

[0, 0, 0] end of KB

Each predicate's facts are a contiguous batch. The ARM maintains an index: predicate ID → DDR3 address of that predicate's batch. Lookup is one index read, then the FPGA filters the batch.

---

3. Inline LLM in the Game Pipeline

3.1 The Key Insight

The LLM does not sit outside the game loop waiting for requests. It is a batch pass inside the frame pipeline, same as fact generation or UtilityAI scoring. Every frame, entities that need new behavior get it through an LLM batch pass that produces new Prolog rules.

3.2 Updated Frame Pipeline

Per frame (16.67 ms budget):

Pass 1: Fact generation

Entity batch → fact batches (predicate-keyed)

Pass 2: State machine evaluation

Entity batch + transition batch + fact batches → updated states

Pass 3: Prolog rule evaluation

Fact batches filtered by rules → valid behavior candidates

Pass 4: LLM inference (inline, on entities flagged for new behavior)

Entity state + fact batches + KB → new rule batches

New rules written to rule batch in DDR3

Immediately available to next pass

Pass 5: UtilityAI scoring

Entity batch + fact batches + behavior batches → winning behaviors

(Uses rules from pass 3 AND new rules from pass 4)

Pass 6: Logic block execution

Entity batch + winning behaviors → modified entities

Pass 7: Envelope processing + trace recording

Pass 8: Beam-casting + rendering

Pass 4 is the LLM. It runs on the same FPGA cores that run every other pass. It processes entities that have been flagged for behavior generation — entities that encountered a situation their current rule set doesn't cover, or entities the designer marked for dynamic behavior.

3.3 When the LLM Fires

Not every entity runs through the LLM every frame. The LLM pass processes only entities with a trigger:

LLM trigger conditions (checked in Pass 3):

1. Rule gap: Prolog evaluation found no matching rule for the entity's current state.

   The entity has no valid behavior. It needs a new rule.

2. Designer flag: The entity's behavior set includes a "generate" behavior

   that delegates to the LLM when it wins the UtilityAI scoring.

3. Novel situation: The entity's fact set contains a combination

   that no existing rule covers. Detected by Prolog as zero path coverage.

4. Periodic refresh: The entity is scheduled for behavior update

   every N frames (configurable per entity type).

Most entities most frames: skip pass 4 entirely. The LLM fires only when needed. Typical frame: 0-10 entities out of 100,000 trigger the LLM.

3.4 LLM Batch Structure

The LLM receives a batch of entity contexts and produces a batch of new rules:

LLM input batch:

[F=llm_input, count=triggered_entities, depth=20]

Each input:

[entity_id, 0] pair 0

[current_state, 0] pair 1

[fact_0, 0] pair 2: most relevant fact (predicate ID + arg)

[fact_1, 0] pair 3

[fact_2, 0] pair 4

…

[fact_15, 0] pair 17: up to 16 relevant facts

[goal_type, 0] pair 18: what kind of rule is needed

[context_hash, 0] pair 19: hash of broader context for pattern matching

LLM output batch:

[F=llm_output, count=triggered_entities, depth=10]

Each output:

[entity_id, 0] pair 0

[new_rule_head, 0] pair 1: predicate ID for the new rule

[body_pred_0, 0] pair 2: first body predicate

[body_pred_1, 0] pair 3

[body_pred_2, 0] pair 4

[body_pred_3, 0] pair 5: up to 4 body predicates

[action_id, 0] pair 6: what action to take if rule passes

[confidence, 0] pair 7: LLM's self-assessed confidence

[source_pattern, 0] pair 8: which training pattern was used

[verification_status,0] pair 9: 0=unverified, set by Prolog in next pass

The LLM output is a rule expressed as integer IDs. Predicate IDs, not strings. Action IDs, not code. The output plugs directly into the rule batch for the next Prolog evaluation pass.

3.5 Prolog Verification of LLM Output

The LLM output goes through Prolog verification before it affects any entity:

Pass 4a: LLM generates rule batch (neural inference)

Pass 4b: Prolog verifies rule batch (batch filter)

Verification checks:

1. Do all body predicate IDs exist in the KB?

Filter: for each body_pred, check if F=body_pred batch exists with count > 0

2. Is the rule logically consistent with existing rules?

Filter: check that new rule's head+body combination doesn't contradict existing rules

3. Is the action ID valid for the entity's current state?

Filter: check action_id against the entity's behavior set

4. Does the confidence meet threshold?

Compare: confidence >= minimum_confidence_threshold

If all pass: verification_status = 1 (verified)

If any fail: verification_status = 0 (rejected, rule discarded)

Only verified rules enter the active rule set. Unverified rules are dropped. The entity falls back to its existing behavior. No hallucinated behavior can reach an entity because the verification pass structurally prevents it.

3.6 The LLM Produces Data, Not Code

The LLM never generates executable instructions. It generates VFR batches that describe rules. The rules are integer tuples: predicate IDs and action IDs. These are data that Prolog evaluates. The evaluation is the same batch filtering that already runs every frame.

There is no code generation. There is no compilation. There is no injection risk. The LLM writes integers to a rule batch. Prolog reads the rule batch. If the integers form a valid rule, the rule activates. If they don't, the rule is discarded. An LLM that outputs nonsense integers produces rules that fail verification and are discarded silently.

---

4. LLM Inference on VFR Hardware

4.1 Forward Pass as Batch Operations

A transformer forward pass consists of:

1. Embedding lookup: index into weight matrix. One batch read.
2. Attention: Q $\times$ K^T matrix multiply, scale, softmax (integer approximation), multiply by V.
3. Feedforward: two matrix multiplies with activation function between.
4. Layer norm: mean and variance (integer), scale and shift.
5. Repeat for each layer.
6. Final projection: matrix multiply to vocabulary logits.

Every step is MUL, ADD, SHR on integer arrays. The attention softmax uses an integer approximation (lookup table, same as Perlin fade function). Layer norm uses integer mean (sum and reciprocal multiply) and integer variance (sum of squared differences and reciprocal multiply).

4.2 Model Size and FPGA Capacity

For inline game use, the model is small. 100M parameters at i32 = 400 MB. This fits in DDR3 with room for everything else. The FPGA loads weight tiles into core BRAMs, processes batch matrix multiplies, writes activations back.

A 100M parameter model with 12 layers, hidden dimension 768, running on 30 FPGA cores at 150 MHz:

Per token inference:

Embedding: 768 lookups = negligible

Per layer: 2 attention matmuls + 2 feedforward matmuls

Each matmul: $768 \times 768 = 589{,}824$ multiply-accumulates

4 matmuls per layer: 2,359,296 MACs

12 layers: 28,311,552 MACs

Final projection: $768 \times$ vocab_size MACs

Total per token: ~30M integer MACs

At 150 MHz $\times$ 30 cores (each doing 1 MAC/cycle): 4.5 GMACs/s

Time per token: 30M / 4.5G = 6.67 ms

Per frame budget for LLM: ~5 ms (from headroom)

Tokens per frame: ~0.75 tokens

Less than one token per frame at this model size on the Zynq-7020. That's too slow for real-time generation. But the LLM only fires on triggered entities (0-10 per frame), and a new rule is only 5-10 tokens. So:

New rule: ~8 tokens

Time to generate: 8×6.67 ms = 53 ms = ~3 frames

Strategy: spread LLM inference across multiple frames.

Frame N: entity triggers LLM, begin inference

Frame N+1: continue inference (entity uses existing behavior)

Frame N+2: continue inference

Frame N+3: inference complete, Prolog verifies, new rule active

The entity doesn't freeze while the LLM works. It continues using its existing behavior for 3 frames, then seamlessly transitions to the new behavior. At 60fps, 3 frames is 50ms. Imperceptible to the player.

4.3 Scaling with Larger FPGAs

Hardware Cores MACs/s ms/token Tokens/frame

———————— ———— ———— ———————— ————————————

Zynq-7020 (30) 30 4.5G 6.67 0.75

Zynq-7045 (130) 130 19.5G 1.54 3.2

Zynq-7100 (270) 270 40.5G 0.74 6.7

UltraScale+ (400) 400 60.0G 0.50 10.0

On the Zynq-7100, the LLM generates a new rule in under one frame. On UltraScale+, multiple rules per frame. Scaling is linear with core count because matrix multiply is embarrassingly parallel.

4.4 Larger Models

For more capable inference (1B parameters), the model is 4 GB. Still fits in DDR3 on larger boards with 4-8 GB. Inference time scales linearly with parameter count. A 1B model on Zynq-7100 takes ~7.4 ms per token — still viable for multi-frame generation.

The architecture doesn't change. More parameters means more weight tiles to stream through the cores. Same instructions, same batch format, same pipeline integration.

---

5. The Knowledge Base as Live Game State

5.1 KB and Entity State Are the Same Thing

The entity batch IS part of the knowledge base. Entities are facts. An entity at position (100, 200) with health 80 is a set of facts:

F=position: [entity_42, 100, 200, …]

F=health: [entity_42, 80, …]

F=state: [entity_42, combat, …]

These facts are generated from the entity batch every frame (Pass 1). The KB doesn't duplicate entity state — it IS entity state, expressed as predicate-keyed batches.

5.2 Persistent Facts Across Frames

Some facts persist across frames without regeneration:

Persistent KB facts:

F=has_item: [entity_42, sword_7, …] (until item dropped)

F=knows_about: [entity_42, secret_door, …] (permanent knowledge)

F=allied_with: [entity_42, entity_89, …] (until relationship changes)

These live in DDR3 as separate batches from the per-frame generated facts. The ARM manages their lifecycle. They enter the KB when events occur and leave when invalidated.

5.3 LLM-Generated Rules Persist

When the LLM generates a new rule for an entity, that rule persists in the rule batch until explicitly removed or superseded:

Frame 100: Entity 42 encounters novel situation, LLM generates rule

Frame 103: Rule verified, enters active rule set

Frame 104+: Rule evaluates every frame as part of normal Prolog pass

Frame 5000: Designer pushes updated rule set, LLM rule superseded

The LLM doesn't regenerate the rule every frame. It generates once, Prolog verifies once, and the rule lives in the batch as data. Same as designer-authored rules. The entity doesn't know whether its rules came from a designer or an LLM. They're all integer tuples in a rule batch.

5.4 The KB Grows During Play

As the game runs, the KB accumulates:

- Facts from entity state (regenerated per frame)
- Persistent facts from game events (items, relationships, discoveries)
- Rules from designers (loaded at scene start)
- Rules from LLM inference (generated during play)
- Provenance for all of the above

The ARM manages KB size. Cold facts (not accessed in N frames) are evicted to SD card. Hot facts stay in DDR3. The FPGA only ever processes the hot set.

---

6. Triveritas as Batch Operations

6.1 Three Filter Passes

Every claim entering the KB is evaluated on three dimensions:

Logical validity (L):

Filter the KB for contradictions:

For the new fact [F=pred, args…]:

Scan existing facts with same F

Compare args for direct contradiction

If contradiction found: L = fail, store contradicting fact ID

If no contradiction: L = pass

One batch filter. Integer comparison on predicate batches.

Mathematical coherence (M):

Check value ranges and type compatibility:

For each argument in the new fact:

Compare against known valid ranges for that predicate's argument position

If out of range: M = fail

If in range: M = pass

If predicate has no range constraints: M = untested

One batch comparison per argument. Bounds checking.

Empirical anchoring (E):

Check provenance chain:

Count source IDs attached to the fact

Check verification level of each source

If 0 sources: E = fail (no provenance)

If 1 source with low verification: E = weak

If 3+ sources with medium+ verification: E = pass

Count and compare on provenance fields. Integer operations.

6.2 Classification

All three pass: → knowledge (high confidence)

Two of three pass: → strong opinion (usable with caveat)

L pass, M pass, E fail: → rationalist trap (logical but ungrounded)

E pass, L fail: → empiricist trap (observed but inconsistent)

L pass, M fail, E fail: → scholastic trap (argument without evidence or math)

The classification is a 3-bit bitmask: `[L, M, E]`. Eight possible combinations. Each maps to a confidence category. One byte of metadata per fact.

6.3 Inline with Frame Pipeline

Triveritas evaluation runs as part of Pass 4b (Prolog verification of LLM output):

Pass 4a: LLM generates rule candidates (batch of integer tuples)

Pass 4b: Prolog verification

Step 1: L check — scan for contradictions

Step 2: M check — validate argument ranges

Step 3: E check — verify provenance depth

Step 4: Classify — combine L/M/E into confidence category

Step 5: Gate — reject facts below threshold

Five batch operations. All integer comparison and counting. Same ISA, same cores, same frame pipeline.

---

7. Scales Method as Batch Filter

7.1 Materiality Gate

Before the LLM processes a triggered entity, the Scales Method checks if the trigger is worth processing:

For each triggered entity:

scope = (affected_entities * 256) / total_entities (fixed-point percentage)

if scope < 13: (5% of 256)

severity = negligible

skip LLM pass for this entity

if scope >= 13 AND scope < 51: (5-20%)

severity = minor

process with low token budget

if scope >= 51 AND scope < 128: (20-50%)

severity = significant

process with normal token budget

if scope >= 128: (50%+)

severity = critical

process with high token budget, prioritize

One multiply, one shift, a few compares. Filters the LLM input batch before inference starts. Entities with negligible scope don't waste inference cycles.

7.2 Fruit of the Plant Validation

After a new rule has been active for N frames, check if it achieved its goal:

For each LLM-generated rule active for > 60 frames:

Compare entity state before rule activation to current state

Did the entity's goal metric improve?

if improved:

fruit = strong

increase rule confidence

keep rule

if unchanged:

fruit = neutral

maintain rule confidence

keep rule (may be situational)

if worsened:

fruit = fails

decrease rule confidence

if confidence < threshold: remove rule

flag entity for new LLM generation

Batch comparison of entity metrics at two timepoints. Integer subtract and compare. The same operations used everywhere else.

---

8. Domain Eating as VFR Batch Loading

8.1 Adding a Domain

Adding a new knowledge domain to the system:

Step 1: Obtain source material (text files, code, structured data)

Step 2: Run domain parser (Zig program on ARM)

Step 3: Parser outputs VFR fact batches with provenance

Step 4: Validate consistency (Prolog verification pass on FPGA)

Step 5: Load validated batches into KB region in DDR3

Step 6: New facts immediately available to all Prolog queries

No neural network retraining. No weight updates. The KB grows. New predicates get new F values. New rules reference new predicates. The FPGA processes them identically to existing facts.

8.2 Parser Output Format

Every domain parser produces the same output — VFR fact batches:

Zig stdlib parser:

Input: Zig source files

Output:

[F=function_sig, count=N, depth=8] function signatures

[F=type_def, count=N, depth=6] type definitions

[F=builtin, count=N, depth=5] builtin functions

All with provenance: source file hash, byte offset, Zig version

English parser:

Input: text documents

Output:

[F=word, count=N, depth=5] word entries with POS tags

[F=phrase, count=N, depth=8] phrase patterns

[F=definition, count=N, depth=6] word definitions with source

All with provenance: document hash, byte offset, edition

Medical parser:

Input: medical reference databases

Output:

[F=symptom, count=N, depth=6] symptom descriptions

[F=treatment, count=N, depth=7] treatment protocols

[F=interaction, count=N, depth=5] drug interactions

All with provenance: database version, entry ID, review date

Different domains. Same batch format. Same integer operations to query. Same Prolog to reason over. The FPGA doesn't know it switched from game AI to medical knowledge. It's filtering integer batches.

8.3 Cross-Domain Queries

Because all domains share the VFR batch format and Prolog evaluation:

A game entity that is also a medical simulation:

F=has_condition: [npc_42, flu, …]

F=symptom: [flu, fever, …] (from medical KB)

F=treatment: [flu, rest, …] (from medical KB)

F=behavior: [npc_42, seek_healer, …] (from game rules)

Prolog connects them:

should_seek_healer(Entity) :-

has_condition(Entity, Condition),

symptom(Condition, Symptom),

severity(Symptom, high),

treatment(Condition, Treatment),

available(Treatment, false).

The medical facts and the game facts are both integer batches with different F values. Prolog unifies across them through shared argument values (entity IDs, condition IDs). No special cross-domain mechanism. Integer comparison doesn't care what domain the integers came from.

---

9. No Hallucination by Construction

9.1 The Structural Guarantee

The LLM produces integer tuples representing candidate rules. These tuples enter the Prolog verification pass. Verification checks every predicate ID against the KB. Every argument against known ranges. Every action against the entity's valid action set.

A hallucinated rule would contain a predicate ID that doesn't exist in the KB, or arguments outside valid ranges, or an action the entity can't perform. Verification rejects it. The integers are discarded. The entity continues with existing behavior.

There is no path from "LLM outputs nonsense integers" to "entity performs hallucinated behavior." The verification pass is structurally between them.

9.2 What Failure Looks Like

When the LLM produces a bad rule, the system doesn't crash, doesn't produce weird behavior, doesn't hallucinate. It produces nothing. The entity falls back to its existing

rule set. The trigger remains active. Next time the LLM fires for this entity, it gets the rejection reason in its context and tries a different approach.

Frame 100: Entity triggers LLM

Frame 103: LLM produces rule with invalid predicate ID

Frame 103: Prolog rejects (predicate F=999 has count=0 in KB)

Frame 103: Entity continues existing behavior

Frame 160: Entity triggers LLM again (periodic refresh)

Frame 163: LLM produces valid rule (learned from rejection context)

Frame 163: Prolog verifies, rule activates

The worst case is: the entity keeps using its existing behavior because the LLM can't produce a valid new rule. This is safe. It's the same behavior the entity had before the LLM existed.

9.3 Path Coverage as Query

The system can report what it knows and what it doesn't:

For any query about entity behavior:

path_count = count of rules matching entity's current fact set

0 paths: "No rules cover this situation" (LLM trigger)

1 path: "One rule applies" (fragile, flag for review)

3+ paths: "Multiple rules agree" (high confidence)

Integer count of matching entries in the rule batch. The system doesn't guess. It counts.

---

10. Session Architecture

10.1 Sessions as KB Views

A session is a filter on the KB, not a separate conversation:

Session: [V: i32, R: i8]

V = session_id

R = session_type (0=game, 1=editor, 2=network, 3=debug)

Multiple sessions can read and write the same KB simultaneously. Each session has an active context filter (which entity types, which predicates, which version). Facts created in one session are visible to all sessions through shared predicates.

10.2 No Degradation

The KB persists and grows. There is no context window to overflow. There is no attention degradation over distance. Prolog matches on predicate IDs and argument values, not on position in a token sequence. A fact from frame 1 is equally accessible as a fact from frame 100,000.

10.3 Memory Management

Hot facts: DDR3, accessed by FPGA within 1 DMA cycle

Warm facts: DDR3, not in current FPGA load, accessed by ARM within 1 read

Cold facts: SD card, loaded on demand by ARM

Eviction: LRU by access count and recency

eviction_score = age * 256 / (access_count + 1)

if eviction_score > threshold: move to SD card

Nothing is deleted. Cold facts can always be reloaded.

---

11. Complete Data Type Summary

Every type in the entire system:

Type V (i32) R (i8)

——— ——————— ———————

Number value VFR remainder

Character Unicode codepoint script family

Term type+value (depth carries provenance)

Fact predicate args (depth carries provenance)

Rule head+body refs (depth carries body count)

Entity field field value field remainder

Pixel packed RGB alpha

Audio sample PCM amplitude channel ID

SVDAG node child offset child mask

Instruction encoded opcode+ops (same format as data)

Weight trained weight quantization remainder

Activation layer output quantization remainder

Session session ID session type

Batch header [F: i16, count: u32, depth: u8] = 7 bytes

One representation. One format. One instruction set processes all of them. The meaning comes from F, from position in the batch, from where the batch is routed. The bits are always `[i32, i8]`.

---

12. Frame Pipeline — Final Form

Per frame (16.67 ms at 60 FPS):

Pass 1: Fact generation

Entity batch → predicate-keyed fact batches

All integer: compare fields, emit to predicate batches

Pass 2: State machine evaluation

Fact batches + transition batch → state updates

All integer: compare, conditional write

Pass 3: Prolog rule evaluation

Fact batches filtered by rule batch → valid candidates

Check for LLM triggers (zero path coverage)

All integer: batch filter, compare, count

Pass 4a: LLM inference (triggered entities only, may span frames)

Entity context batch → candidate rule batch

All integer: matrix multiply-accumulate on weight batches

Pass 4b: Prolog verification of LLM output

Candidate rules checked against KB

Triveritas: L/M/E three filter passes

Scales: materiality gate (scope check)

All integer: compare, count, threshold

Pass 5: UtilityAI scoring

Fact batches + behavior batches + verified rules → winning behaviors

All integer: multiply chain, fixed-point normalization

Pass 6: Logic block execution

Winning behaviors → entity field modifications

All integer: ALU operations on entity batch

Pass 7: Envelopes + traces

Time-bounded modifiers + decision recording

All integer: multiply, compare, store

Pass 8: Beam-casting + rendering

SVDAG drill + Gouraud + Perlin → pixel buffer

All integer: shift, mask, compare, multiply, lookup

Pass 9: Upscale + UI composite

Edge-aware upscale + Clay layout → final framebuffer

All integer: compare metadata, LERP, pixel write

Every pass. Every operation. Every data type. Integer batches processed by 35 instructions.

---

13. Summary

Component	VFR Representation	Processing
Text/strings	V=codepoint, R=script family	Batch compare, filter by R
Terms	Depth=5: type, value, source, offset, version	Batch transform and filter
Facts	F=predicate_id, depth includes provenance	Batch filter (Prolog evaluation)
Rules	Head + body predicate references	Batch lookup and chaining
LLM weights	V=weight, R=quantization remainder	Matrix multiply-accumulate
LLM activations	V=activation, R=quantization remainder	Multiply, add, shift
Knowledge base	Stream of predicate-keyed fact batches	Filter, count, compare
Triveritas	Three filter passes: L, M, E	Compare, count, threshold
Scales	Scope percentage, materiality gate	Multiply, shift, compare
Sessions	V=session_id, R=session_type	KB view filter
Hallucination	Structurally impossible	Verification between generation and use

Component	VFR Representation	Processing
Domain eating	New parser → new fact batches → KB grows	No retraining, load and query

---

LLM-Prolog on VFR Hardware — Inline Intelligence Specification v1.0

Companion document to ISA v1.0, Compiler v1.0, Silo-VFR Mapping v1.0, Motherboard v2.0, Rendering Pipeline v2.0, FPGA Implementation v1.0, System Integration v1.0, and The Philosophy of Q-Computation

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper LEX-12-2026